

Data exploration toolkit for cultural data

Chiata Livio

Stefano Rapisarda

Barbara Romero Ferron

2025-05-19

Table of contents

Welcome!	5
Your Trainers	6
Schedule	7
 I Preparation	 8
How to prepare for the workshop	9
Running the analysis locally on your own computer	9
Installing Python	9
Downloading data and analysis code	9
Start Google Colab	10
(alternative) Open Jupyter notebook	10
 Tips on using generative AI	 11
Study case	11
Line-by-Line Explanation of What the Code Does	13
List of Fundamental Programming Concepts Present in the Code	14
References for Each Programming Concept	15
Glossary of Technical Programming Terms	16
Handling errors	16
 II Data Workflow	 18
1 Organising your data	19
2 Introduction to Python	20
Getting familiar with Jupyter Notebook	21
Jupyter Notebook cells	22
Main programming concepts	23
Variables and data types	23

Sequences of objects	24
Data types: Dictionaries	25
Functions	26
Methods	27
Loops	29
Conditional statements	32
Packages	34
3 Reading data	36
4 Exploring data	38
Previous steps	38
5 Cleaning data	42
Previous steps	42
6 Analysing data	48
Previous steps	48
Data Analysis	50
Summary	60
Databases and Data Analysis	60
General data analysis workflow	60
What's next?	62
Glossaries	63
Data Glossary	63
Controlled vocabularies	63
Data cleaning	63
Data harmonisation	63
Data models	63
Enrichment	63
Graph database	63
ID	64
Normalisation	64
Relational database	64
Programming glossary	64
Code	64
csv	64
DataFrame	64
Initialisation	65
Library	65

Loop	65
Object	65
Package	65
Series	66
Variable	66
Resources	67
Research Support	67
Datasets	67
Programming	68

Welcome!

From: Sonya Rapoport, Anasazi Series, (pencil, prismacolor, colored typewriter, and computer print on continuous-feed computer paper; 2 folios, 15 and 14 pages each), 1977 [ref: Leslie Jones, CODED: Art at the Dawn of the Computer Age, 1960-1980, Los Angeles County Museum of Art, Los Angeles, CA, Fall 2021]

Welcome to the Cultural Data Exploration Toolkit! Throughout this workshop, you'll explore the nuances of data-driven analysis, from constructing your dataset and formulating research inquiries to learning data visualisation techniques. Along this journey, you'll contemplate the creation of personalised data models tailored to your research queries, navigate potential biases within datasets, and, importantly, learn how to effectively interrogate, explore, and analyse gathered information to generate visualisations.

Your Trainers

Schedule

Time	Activity
13:00	Welcome - Introduction
13:20	Group activity: structuring data
13:50	Discussion
14:00	Break
14:10	Introduction to data toolkit
14:50	Set up analysis environment
15:00	Break
15:10	Group activity: Data Exploration I
15:30	Discussion
15:40	Data Exploration II
16:40	Recap & Questions

Part I

Preparation

How to prepare for the workshop

The workshop contains both interactive and lecture-like sessions. The first part of the workshop will be focused on concepts and its interactive experience does not require any prior knowledge. In the second part of the workshop, you will go through the main concepts of exploratory data analysis, with data analysis being performed via programming (specifically Python). As this is not a programming workshop and it is open to all researchers (independent of their programming background), we will do our best to explain what the programming does and how it works, but we will not teach you how to program, focusing more on the data analysis concepts and fundamental steps. With that in mind, you can decide to follow the class in three different ways:

- **“Sit and enjoy the show”:** focus on understanding and awareness. The instructor will guide you through the main concepts of programming and data analysis. You can later download code and data and (following the detailed description on this page) and play with data analysis yourself, at your own pace;
- **“Following and clicking”:** you can go through the analysis step by step together with the instructor. Data will be shared on Google drive and you will be guided through the analysis on Google Colab. You would not need to install anything, just having a Google account;
- **“Give me all!”:** you want to run the entire analysis in your own laptop, this is the hardest (but not as much as it seems) path! In this case follow the installation instructions in the following sections.

Running the analysis locally on your own computer

Installing Python

1. Install python and jupyter notebook. For installation and setup we point at the “Introduction to Python & Data” [Installation & Setup](#) page;

Downloading data and analysis code

1. Check the available Dataset in **Resources**, choose the one you are interested in, and follow the instructions in their pages for the download;

2. Extract the content of the zip file (in case the downloaded file is a zip file):
 - If you are working on **Windows**, you can right-click on the zip file and, from the context menu, click on “Extract All”. A dialog box will appear asking where you want to extract the files. You can either use the default folder (same location as the ZIP file) or click “Browse” to select a different folder. We recommend to create a new folder called “cultural_data_explorative_analysis” and extract your files there;
 - If you are working on **macOS**, you just need to double click on the zip file. Your computer will automatically create an unzipped directory with the same name of the zip file. We recommend to rename this directory “cultural_data_explorative_analysis”;

Start Google Colab

1. Put the unzipped downloaded file containing your data in a Google Drive folder;
2. Click on the folder in the navigation panel on the left and click on the button “New” on top of the navigation panel;
3. From the drop down menu, select first “More” and then “Google Collaboratory”. A collaborative notebook will open and you are set up for using Python.

(alternative) Open Jupyter notebook

1. Open Anaconda Navigator from the Start Menu (**Windows** Users) or from the Applications folder or Launchpad (**macOS** Users);
2. Click on “Launch” under Jupyter Notebook. This will open a web browser;
3. Navigate to the folder containing your downloaded unzipped data;
4. Click on the notebook file (.ipynb) to open it.

Tips on using generative AI

We encourage the use of generative AI as a valuable learning tool for those looking to learn Python—whether you’re a programming beginner or someone seeking to improve your coding skills.

To maximize the effectiveness of generative AI, it’s essential to tailor your prompts to the specific information you want to access. Thoughtful prompting can significantly enhance the quality of the responses you receive.

When posing questions to generative AI, keep the following tips in mind:

Be Specific and Unambiguous: Clearly articulate your question to avoid confusion. **Break Down Complex Problems:** Divide general inquiries into smaller, more manageable questions. **Don’t Settle for the First Response:** Explore further by asking follow-up questions to deepen your understanding. More specifically, consider these best practices:

Use clear, concise, and unambiguous language. Assign a role to the AI (e.g., software engineer, teacher, data analysis expert). Provide context and examples to enhance understanding. Specify the desired output format for clarity. Evaluate and refine your input based on the responses you receive. By following these guidelines, you can harness the full potential of generative AI in your learning journey!

Study case

Let’s consider the very first piece of Python code you will find in this course:

```
import pandas as pd

data_file = 'data/data.csv'
df = pd.read_csv(data_file)

print(type(df))
```

Let’s also assume that you have no idea about how Python works, so you will use generative AI copying and pasting this piece of code and ask “explain this”. This is what you might get:

Explanation of the Code:

```
import pandas as pd

data_file = 'data/data.csv'
df = pd.read_csv(data_file)

print(type(df))
```

This code performs the following tasks:

1. Importing the pandas library:

- The line `import pandas as pd` imports the pandas library, which is used for data manipulation and analysis in Python. The alias `pd` is a common convention that makes it easier to reference pandas functions.

2. Defining a file path:

- The variable `data_file` is assigned the string `'data/data.csv'`, which represents the path to a CSV (Comma-Separated Values) file named `data.csv`. This file is located in a subdirectory called `data`, relative to the current working directory.

3. Reading the CSV file:

- The line `df = pd.read_csv(data_file)` uses the `read_csv()` function from the pandas library to read the CSV file specified by the `data_file` variable. The data from the CSV is loaded into a pandas DataFrame, which is a 2-dimensional labeled data structure similar to a table.
- The variable `df` now holds this DataFrame, which contains the data from the CSV file.

4. Printing the type of the DataFrame:

- The line `print(type(df))` prints the type of the variable `df` to the console. This is useful for confirming that the data was successfully loaded into a DataFrame.

This is a comprehensive explanation of what that piece of code does, but there may still be some room for improvement, especially if we are absolute beginners and we are not familiar with some of the technical language used in the description. Furthermore, not only we want to know what the code does, but we also want to take the opportunity to learn about programming. Let's try with another prompt:

Act as a software engineer, data scientist, and teacher. I am a bachelor student with no-prior knowledge in programming. I want to understand what code does and general programming concepts. Explain the following piece of code with the following output: 1) explanation of what the code does in few words, 2) line by line explanation of what the code does, 3) a list of

fundamental programming concepts present in the piece of code with further example, 4) for each programming concept, three references (website, books, tutorials, etc) to deepen my knowledge. Finally, create a glossary of the technical programming terms in a format that I can copy and paste in my notes. [attach here code in between double quotes]

This is the response:

Explanation of the Code:

```
import pandas as pd

data_file = 'data/data.csv'
df = pd.read_csv(data_file)

print(type(df))
```

This code imports the pandas library, reads a CSV file into a DataFrame, and prints the type of the DataFrame to confirm successful data loading.

Line-by-Line Explanation of What the Code Does

```
import pandas as pd
```

- **What it does:** This line imports the pandas library and assigns it the alias `pd`.
- **Purpose:** The alias `pd` is used for convenience, making it easier to reference pandas functions throughout the code.

```
data_file = 'data/data.csv'
```

- **What it does:** This line creates a variable named `data_file` and assigns it the string value `'data/data.csv'`, which is the path to a CSV file.
- **Purpose:** The variable holds the location of the file, making it easy to access and modify later.

```
df = pd.read_csv(data_file)
```

- **What it does:** This line uses the `read_csv()` function from the pandas library to read the CSV file specified in the `data_file` variable and stores the resulting DataFrame in the variable `df`.

- **Purpose:** It loads the data from the CSV file into a DataFrame, which allows for easy data manipulation and analysis.

```
print(type(df))
```

- **What it does:** This line prints the type of the variable `df` to the console.
 - **Purpose:** It confirms that the data was loaded correctly by displaying the type of object `df` holds, which should be a pandas DataFrame.
-

List of Fundamental Programming Concepts Present in the Code

- **Importing Libraries/Modules:**
 - Description: Using external libraries to enhance the functionality of your code.
 - Example: `import pandas as pd` brings in the pandas library for data analysis.
 - **Variables:**
 - Description: Named storage locations for data in a program.
 - Example: `data_file = 'data/data.csv'` creates a variable named `data_file`.
 - **Data Types:**
 - Description: Categories of data that tell the compiler or interpreter how the programmer intends to use the data.
 - Example: The variable `data_file` is a string, while `df` becomes a DataFrame.
 - **Functions:**
 - Description: A reusable block of code that performs a specific task.
 - Example: `pd.read_csv(data_file)` is a function call that reads the CSV file.
 - **Print Statements:**
 - Description: Used to display output to the console.
 - Example: `print(type(df))` displays the type of the DataFrame.
-

References for Each Programming Concept

Importing Libraries/Modules

1. [W3Schools - Python Modules](#)
2. [Python Documentation - Modules](#)
3. **Book:** “Automate the Boring Stuff with Python” by Al Sweigart

Variables

1. [W3Schools - Python Variables](#)
2. [Real Python - Python Variables](#)
3. **Book:** “Python Crash Course” by Eric Matthes

Data Types

1. [W3Schools - Python Data Types](#)
2. [GeeksforGeeks - Data Types in Python](#)
3. **Book:** “Head First Python” by Paul Barry

Functions

1. [W3Schools - Python Functions](#)
2. [Real Python - Defining Functions in Python](#)
3. **Book:** “Fluent Python” by Luciano Ramalho

Print Statements

1. [W3Schools - Python Print Function](#)
 2. [Real Python - The Python Print Function](#)
 3. **Book:** “Learning Python” by Mark Lutz
-

Glossary of Technical Programming Terms

****Glossary of Technical Programming Terms****

- ****Importing Libraries/Modules****: The process of including external libraries in a program.
- ****Variables****: Named storage locations in a program that hold data values.
- ****Data Types****: Categories of data that determine the kind of operations that can be performed on them.
- ****Functions****: Reusable blocks of code that perform specific tasks; they can take inputs and return outputs.
- ****Print Statements****: Commands used to display output on the console or standard output.

As you may notice, the generative AI output is now much more comprehensive. In this way we turned few lines of code in a small learning experience. There is still room for improvement, for example the concept of library (or package) is not explained and the examples provided are not more than those same lines of code you asked to explain. You may ask generative AI to be even more explicit in the explanation of technical terms and to provide different examples.

Handling errors

Errors are an inherent part of programming. When you run a program, encountering an error often means that something went wrong, causing the program to stop functioning. The root cause of an error can range from simple syntax issues, such as typos, to more complex problems. Regardless of the cause, diagnosing the issue can be time-consuming. Fortunately, generative AI can help streamline this troubleshooting process.

When you encounter an error, you will typically see an error message—text that explains why the error occurred. For those new to programming, these messages can seem cryptic. However, they contain valuable clues that can assist you in identifying the problem. If you lack programming experience, generative AI can help you understand the error and suggest corrective actions.

Here are some general tips to consider when dealing with errors:

- **Identify the Location of the Error:** Error messages often indicate the line of code where the problem occurred. Before that line, check if the program runs correctly by dividing your code into smaller chunks and running them separately.
- **Don't Ignore Error Messages:** Pay attention to the type of error message you receive. Use generative AI to explore common causes for that specific error type.
- ***Check Syntax with Generative AI:** Copy and paste the portion of your code that is generating the error into generative AI and ask it to review the syntax for potential typos or mistakes.

- **Provide Specific Context:** When querying generative AI, be explicit about your programming environment. Mention the programming language you are using (even though AI can often recognize it), the libraries involved, the task your program is performing, and any relevant data types you are working with.

Part II

Data Workflow

1 Organising your data

2 Introduction to Python

In this session, we will introduce you to Jupyter Notebooks and guide you through some of the essential programming concepts that form the foundation of working with data. Whether you are completely new to programming or looking to strengthen your understanding, this chapter is designed to help you build confidence as you take your first steps into coding.

This session is very python-specific, but (as you will find out in few lines) Jupyter Notebook is compatible with many programming languages and the programming concepts introduced here are fundamentals of every programming language.

A Jupyter Notebook is an interactive environment where you can write and execute code in small, manageable pieces, known as cells. This allows you to see the results of your code immediately, making it an excellent tool for learning, experimenting, and exploring data. It combines text, code, and the results of that code all in one place, making it a popular choice for data scientists, researchers, and educators.

In this session, we will be using Python, a programming language known for its simplicity and readability, which makes it ideal for beginners. We will cover core programming concepts, such as variables, functions, and loops, and you will learn how to apply these concepts to perform basic data exploration tasks.

Getting familiar with Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text. It supports various programming languages, including Python, R, Julia, and more. However, it is most commonly used with Python.

Jupyter Notebook provides an interactive computing environment where you can write and execute code in a series of cells. Each cell can contain code, markdown text, equations, or visualizations. You can run individual cells or the entire notebook to see the output of the code and the results of any computations.

The name “Jupyter” is a combination of three programming languages: Julia, Python, and R, which were the first three languages supported by the Jupyter project. It was originally developed as part of the IPython project (hence the name “Jupyter”), but has since evolved into a language-agnostic tool that supports multiple programming languages.

In the context of Python, Jupyter Notebook is a popular tool for data analysis, scientific computing, machine learning, education, and research. It allows users to write, test, and document Python code in an interactive and visually appealing manner, making it a valuable tool for both beginners and experienced programmers alike.

Jupyter Notebook cells

Jupyter notebook cells can be either code, markdown, or raw. For the simple purpose of programming and writing text, ignore the raw option. You can easily shift between code and markdown selecting the cell, pressing Esc, and then M for markdown or Y for code.

Markdown is a language for formatting text, it allows you to quickly and easily create formatted documents using simple and intuitive syntax. This current cell and any other cell displaying text in this notebook, is written in markdown. You can learn the basics of markdown syntax in few minutes reading [here](#) or simply looking at the content of the text cells in this notebook and see what happens when you select them and run them.

You can tell if your cell selected cell is a code cell because you will see square brackets on its left ([]:).

If you want to delete a cell, use Esc + DD (press Esc and then d twice)

WARNING: If your code cell has *empty* squared brackets, it means it has not been run YET.

Main programming concepts

There are some programming concepts that are common to all programming languages and can be found in any program:

- variables and data types;
- sequences of objects;
- functions;
- loops;
- conditional statements;
- packages (also called libraries or modules)

Variables and data types

In programming a variable is a container for a value. This value can either be a number, a string (a word), or any other type of programming object (we will talk about other possible objects later). Let's initialise (define for the first time) some variables:

```
name = 'Stefano'  
favourite_planet = 'Saturn'  
birth_day = 6
```

In the previous cell we stored the word *Stefano* into the variable `name`, the word *Saturn* into the variable `favourite_planet`, and the value 6 into the variable `birth_day`. From now on, every time we need to use one of these values in our programming, we just need to digit its corresponding variable name.

In Jupyter notebooks, if you want to check the value contained in a variable (so its content), you can simply run a cell with the variable name inside:

```
name
```

```
'Stefano'
```

```
birth_day
```

6

```
name  
birth_day
```

6

As you can notice, when you write different variable names in the same cell, only the last one will be printed on the screen.

Sequences of objects

We can store single numbers and words inside a variable, but how about we want to store a *sequence* of values or words, or a mix of the two, into a variable? Of course we can, we just need to use a python object called **list**:

```
names = ['Stefano', 'Pippo', 'Alfio', 'Tano']  
ages = [20, 34, 94, 'unknown']
```

```
names[0]
```

```
'Stefano'
```

```
ages[3]
```

```
'unknown'
```

In Python lists are defined listing our sequence of values separated by coma inside square brackets (`variable_name = [... , ... , ...]`). Values stored in a list can be accessed using **indexing**. In python you count items starting from 0, so that the first item in a list has index 0. This means that for accessing the first item in the list **names** we will digit **name[0]**, and to access the last item in the list **ages** we will digit **ages[3]**

You can create lists of *any* object, even lists of lists:

```
info = [names, ages]  
info[0]
```

```
['Stefano', 'Pippo', 'Alfio', 'Tano']
```



```
info[0][0]
```

```
'Stefano'
```

If we want to change a particular value in a list, we first need to access it and then we need to use the operator `=` to specify the new list value. For example, if we want to change 'Stefano' into 'Steve', we would do:

```
info[0][0] = 'Steve'  
info[0]
```

```
['Steve', 'Pippo', 'Alfio', 'Tano']
```

Data types: Dictionaries

In python there are several ways you can store information. We just talked about lists, simple ordered sequences of objects. Another kind of data structure is called **dictionary**. In general a dictionary is a reference or resource that provides information, definitions, or explanations of words, terms, concepts, or objects. A dictionary is usually organised by alphabetically ordered words and by explanations associated to each word. In python a dictionary follows exactly the same organization principle: keyword and value.

```
info_dict = {'name': 'Stefano', 'favourite_number': 6}
```

To define a dictionary we use curled brackets (`{}`) instead of squared brackets. Inside the curled brackets we need to specify couples of key/values separated by commas. To each key we can associate a different python object. Keys need to be unique, while values can be any Python object.

```
info_dict = {'names': names}
```

```
info_dict['names']
```

```
['Steve', 'Pippo', 'Alfio', 'Tano']
```

To access the values contained inside a dictionary you cannot use numerical indices, as you would do for lists. Instead, you must use the name of the key related to the value. In the

previous case the object names (a Python list) is associated with the key 'names'. So, in order to access it, we need to digit `info_dict['names']`.

In a similar way, if you want to change the value related to a key, or create a new key/value couple, you first need to access that value and then to use the '=' sign to assign a new value

```
info_dict['names'] = ['Steve','Josef','Alfonse','Gerrit']
info_dict['names']
```

```
['Steve', 'Josef', 'Alfonse', 'Gerrit']
```

Functions

A function is a python object that performs a single action given some parameters. In python, function names are usually verbs. If variable can be thought as subjects and objects in a sentence, functions are the verbs. Python has already some default functions, functions that are ready to use. [Here](#) you can find the build-in Python functions.

```
result = print(name)
result
```

Stefano

The way a function works is common to all programming languages: you give to the function one or more parameters, the function performs an action, and it returns a result. This happens so fast that, as a matter of fact, you can already think at a function and its parameters as its result. In the previous cell the function `print()` got as an input parameter the variable name and it printed its value on the screen.

```
numbers = [1,2,3,4,5,6,7,8,9,10]
result = sum(numbers)
print(result)
```

55

In the previous cell, we defined a list of values (the first 10 integers), then we used the function `sum` to (guess what??) sum all the numbers in the list, and we stored the result into the variable `result`. We finally printed the result using the function `print()`.

Because we know that variable values are printed automatically in Jupyter notebook cells when they contain the variable name, we could write directly:

```
sum(numbers)
```

55

Indeed `sum(numbers)` represents an operation that returns the value 55 and can be considered equivalent to the value 55 itself, so that when we write it in a cell, we obtain the result printed on the screen.

```
len(numbers)
```

10

The function `len()` is one of the most used function on objects containing many items. Indeed it tell us how many items are contained in that object (i.e. the length of that object). The function `type()` returns the type of a variable:

```
type(numbers)
```

list

How many functions are there? thousands, probably millions. Some of them have very intuitive names (like `print()` and `sum()`), some others have more complicated names. However, every function that can be used in Python comes with its own documentation, explaining which parameters it accepts, which additional options you can specify, and which kind of result you get back when applying it. To find about a function just google “ Python documentation” or ask ChatGPT about that.

Methods

Methods are functions that are object-specific. What does it mean? There are certain operations that can be performed only on a certain type of object. For example, if we consider a function that transforms lower characters into capital letters, it would not make much sense to apply this function to a number.

All objects in python can have their own specific functions and these object-specific functions are called **methods**. To use a method on an object, you need to apply the syntax

`<object_name>.method()`. Do you see the difference compared to a general function syntax? In a general function we have `function(par1,par2,...)`, while in a method we already know that the function, in this case method, will be applied to its object. Therefore, inside the parenthesis we only have additional parameters.

Like functions, methods can accept all kind of parameters, but, of course, their main parameter is the object itself. Let's see some example:

```
name = 'Stefano Rapisarda Arthurus Micaelus'
numbers = [1,2,3,4,5]
```

```
name.split()
```

```
['Stefano', 'Rapisarda', 'Arthurus', 'Micaelus']
```

We initialised two variables: a string made of several words and a list of numbers. The `split()` method (a string-specific function) divides the string into a list of strings according to a separator. If you don't specify any separator (like in our case), white spaces will be considered as separators. Let's see another example:

```
numbers.pop(2)
```

```
3
```

```
numbers
```

```
[1, 2, 4, 5]
```

We initialised two variables: a string made of several words and a list of numbers. The `split()` method (a string-specific function) divides the string into a list of strings according to a separator. If you don't specify any separator (like in our case), white spaces will be considered as separators. The variable `numbers` is already a list and using the method `pop(x)` we can remove the item occupying the 3rd position (index 2). The method affects the list and returns the just removed value.

How can we found about methods if there are so many? Usually a google search can point you at the method or function you need. In general you can always consult python documentation. You will find about string a list methods [here](#) and [here](#), respectively.

Loops

One of the potential of using machines is making them repeating the same operation hundreds, millions, or billions of times.

Let's say I have a list of names and I want to print them on the screen one by one:

```
names = ['James','Martin','Sandra','Paul','Chani']  
print(names[0])  
print(names[1])  
print(names[2])  
print(names[3])  
print(names[4])
```

```
James  
Martin  
Sandra  
Paul  
Chani
```

This did not take us much time, because the names are only 5, but imagine you have a list of 1000 names; in that case printing all the names could take hours. Looking at the previous cell we notice that we use repeatedly the function `print()` using as input the values contained in the list `names`. Every time we need to repeat an operation many times, we can use a **loop**, specifically a for loop:

```
for i in range(5):  
    print(i,names[i])
```

```
0 James  
1 Martin  
2 Sandra  
3 Paul  
4 Chani
```

In the previous cell the same operation (`print()`) is executed 5 times, but at each step, so at each **iteration**, the variable `i` changes, going from 0 to 4, one step at the time.

In order to achieve this result we need to start declaring **for i**. `i` is the variable name acting as a place holder for a value that will change at every step of the iteration. We chose the letter `i`, but you can choose any other name. After **for i**, we need to specify which values `i` can

assume at each iteration. `in range(5)` means that `i` will go from 0 to 4, so it will increase of 1 integer per iteration stopping just before 5. Instead of a range of numbers, we can specify any other object containing several objects in it. In that case, the variable `i` (or whatever you will decide to call it), at each iteration, will be initialized with each value contained in the specified object. Let's see some example:

```
for a in range(12): print(a)
```

0
1
2
3
4
5
6
7
8
9
10
11

```
for value in [0,1,4,5,6,7]:  
    print(value)
```

0
1
4
5
6
7

```
for name in names:  
    print(name)
```

James
Martin
Sandra
Paul
Chani

For *looping* over dictionaries, the concept is the same, but the syntax is a bit different because of the key/value structure of dictionaries:

```
info_dict = {
    'name': 'Stefano',
    'surname': 'Rapisarda',
    'favourite_number': 6,
    'favourite_planet': 'Saturn'
}
for key,value in info_dict.items():
    print(key,':',value)
```

```
name : Stefano
surname : Rapisarda
favourite_number : 6
favourite_planet : Saturn
```

WARNING You noticed that after the for statement, there is an indent of 4 spaces. You can make that indent using the TAB key. That indent tells python that that specific line of code is inside the loop and, therefore, needs to be repeated. When you write code without indents, before or after the loop, those lines will be executed normally, i.e. once, one after the other.

```
print('Beginning of the for loop, we will have 10 iterations')
print('='*72)
for i in range(10):
    print('This is iteration number:',i)
    print('The next iteration will be:',i+1)
    print('End of iteration',i)
    print('-'*62)
print('='*72)
print('End of the loop')
```

Beginning of the for loop, we will have 10 iterations

=====

```
This is iteration number: 0
The next iteration will be: 1
End of iteration 0
```

```
This is iteration number: 1
The next iteration will be: 2
End of iteration 1
```

```
-----  
This is iteration number: 2  
The next iteration will be: 3  
End of iteration 2  
-----
```

```
-----  
This is iteration number: 3  
The next iteration will be: 4  
End of iteration 3  
-----
```

```
-----  
This is iteration number: 4  
The next iteration will be: 5  
End of iteration 4  
-----
```

```
-----  
This is iteration number: 5  
The next iteration will be: 6  
End of iteration 5  
-----
```

```
-----  
This is iteration number: 6  
The next iteration will be: 7  
End of iteration 6  
-----
```

```
-----  
This is iteration number: 7  
The next iteration will be: 8  
End of iteration 7  
-----
```

```
-----  
This is iteration number: 8  
The next iteration will be: 9  
End of iteration 8  
-----
```

```
-----  
This is iteration number: 9  
The next iteration will be: 10  
End of iteration 9  
-----
```

```
=====
```

End of the loop

Conditional statements

We have seen how to store data and information into variables and how to access this information by indexing, so referring to the position of values inside an object. How about selecting information using other criteria? What about if we want to visualize only peoples names if

they are older than 30 or printing the names of towns that start with an ‘s’? To do that in programming we need to use **conditional statements**. Conditional statements are indeed conditions that need to be satisfied in order for something to happen. What is “something”? Whatever action we want: an operation, a printing function, etc.

```
for key,value in info_dict.items():
    if 'favourite' in key:
        print(key,':',value)
    else:
        print('Not interested!')
```

```
Not interested!
Not interested!
favourite_number : 6
favourite_planet : Saturn
```

We used the same for loop to explore dictionaries keys and values, but this time, inside it, we wrote a conditional statement. The syntax for a conditional statement is:

```
if <condition>:
    action
else:
    other_action
```

<condition> is the condition that needs to be satisfied. In this case we want the word ‘favourite’ to be contained inside the key. If this happens, the condition is True and the “action” is performed (in our case, key and value will be printed). If the condition is False, the “other_action” will be performed (in our case, the ‘Not interested’ message will be printed).

You can also make conditions comparing quantities:

```
numbers = [1,2,3,4,5,6,7]
for number in numbers:
    if number < 4:
        print(number,'is smaller than 4')
    elif number > 4:
        print(number,'is larger than 4')
    elif number == 4:
        print(number,'is exactly 4')
```

```
1 is smaller than 4
2 is smaller than 4
3 is smaller than 4
4 is exactly 4
5 is larger than 4
6 is larger than 4
7 is larger than 4
```

Conditional statements may also be combined:

```
for number in numbers:
    if (number < 4) or (number > 4):
        print(number, 'is not 4')
    else:
        print(number, 'must be 4')
```

```
1 is not 4
2 is not 4
3 is not 4
4 must be 4
5 is not 4
6 is not 4
7 is not 4
```

In the previous case we used three conditions that are satisfied if a number is smaller, larger, and equal to 4.

Using loops in combination with conditional statements is particularly useful when it's time to select data. For example, imagine we have data in a table with two columns, one contains years and the other column can be any kind of measurement. In this case, you can use conditional statements to select measurements and very specific time intervals.

Packages

There are millions of functions and objects out there, how can we use them? Python installation does not come with ALL the functions ever written for Python. Functions and objects are usually organized in **packages** (also called libraries or moduli). Each package contains a set of tools specific for certain tasks. There are tools for statistics, machine learning, building website, text-mining, etc. How can we access all these tools? First of all, we need to download

the package into our computer. Usually in the documentation page of the package, there are installation instruction. Once installed, the package needs to be imported.

```
import pandas as pd
from matplotlib import pyplot as plt
```

In the previous cell we imported two packages, pandas and pyplot. When we import something, it is convenient to choose an alias for it, so that, when needed, we don't need to write its entire name. In our case, `pd` will be the alias for pandas.

In the second line we see a slightly different statement. In this case, we import the package pyplot. The package is a sub-package of the massive library matplotlib. Therefore, we need to specify the macro-package containing pyplot. We could also import pyplot in the following way.

```
import matplotlib.pyplot as plt
```

From now on, every time we will need to use a pandas function or object, we just need to specify the alias of the package before the function or object we want to use:

```
df = pd.DataFrame()
```

In the previous case, we initialised a variable called `pd` with a pandas DataFrame. Let's see what happens if we forget to specify `pd`:

```
df = DataFrame()
```

```
NameError: name 'DataFrame' is not defined
```

We obtain an error because Python does not recognize the function name.

3 Reading data

The first thing we need to do is loading the data. This means opening the file where the data is currently stored and transfer that data here, in our working environment. As we are working with Python in this Jupyter notebook environment, this means transferring all the data into a Python object. Which object? There are Python libraries (Python code written by other developers) that have been specifically designed to perform the task of data analysis. One of these libraries, or (“Pythonically” speaking) **packages**, is called **pandas**. We will use one of the many **pandas** functions to read our **.csv** (coma separated values file) file and we will store the information into a **pandas DataFrame**.

<IPython.core.display.HTML object>

```
import pandas as pd
data_file = 'data/data.csv'
df = pd.read_csv(data_file)
print(type(df))
```

<class 'pandas.core.frame.DataFrame'>

We managed to transfer our data into a Python object, specifically a **pandas.core.frame.DataFrame**, or simply (from now on) a **DataFrame**. However, a lot of things can go wrong when going from one format to another, so it is a good idea to have a first look at the data.

<IPython.core.display.HTML object>

```
df.head(10)
```

	Year of arrival at port of disembarkation	Voyage ID	Vessel name	Voyage itinerary imputed port
0	1714.0	16109	Freeke Gally	Bristol
1	1713.0	16110	Greyhound Gally	Bristol
2	1714.0	16111	Jacob	Bristol

	Year of arrival at port of disembarkation	Voyage ID	Vessel name	Voyage itinerary imputed port
3	1714.0	16112	Jason Gally	Bristol
4	1713.0	16113	Lawford Gally	Bristol
5	1714.0	16114	Mercy Gally	Bristol
6	1714.0	16115	Mermaid Gally	Bristol
7	1713.0	16116	Morning Star	Bristol
8	1714.0	16117	Peterborough	Bristol
9	1713.0	16118	Resolution	Bristol

Comparing what we see here with our .csv file it seems that everything went well. We have the data organised in rows and columns. Each column has a name and each row and index. Looking at our data, some values are numbers, some are names and places, some contain html tags, some are NaN. It is not time yet to run data analysis, after having loaded the data we still need to correctly interpret the information it contains, then we need to “clean” it, and after that, finally, we can proceed with some data analysis. This is just the beginning, but the best is yet to come!

4 Exploring data

Previous steps

```
import pandas as pd
data_file = 'data/data.csv'
df = pd.read_csv(data_file)

df.head(5)
```

	Year of arrival at port of disembarkation	Voyage ID	Vessel name	Voyage itinerary imputed port
0	1714.0	16109	Freeke Gally	Bristol
1	1713.0	16110	Greyhound Gally	Bristol
2	1714.0	16111	Jacob	Bristol
3	1714.0	16112	Jason Gally	Bristol
4	1713.0	16113	Lawford Gally	Bristol

Now that we correctly loaded our data in our working environment, it is time to figure out what the data contains. It is always a good idea to look at the dataset [documentation](#) (or metadata) to understand where the data comes from, what is the source of all the different records, how data has been collected, and any other possible data related caveat. Diving into the data documentation is up to you, in this chapter what we want to do is understanding as much as we can from the data itself, looking at its columns, rows, and values. Every dataset tells a story. You may think about it like a person with a long experience, but not really willing to talk (well, some datasets “talk” more easily than others). It is your role in this case to “interrogate” the data, let it to talk, to tell a story and to dive into the details of that story, getting as much information as you can. This also depends on how much you need to know: will you be satisfied by a small “chat” or you need to know all kind of details? Let’s formulate some questions to begin with.

<IPython.core.display.HTML object>

```
df.shape
```

```
(36151, 9)
```

```
solution = 'Our DataFrame contains data distributed in 36151 rows and 9 columns. '  
question_box(solution=solution)
```

```
<IPython.core.display.HTML object>
```

It is a quite big dataset. Shall we care about how big is our dataset? We should as this may affect our analysis. For example, if we implement a scientific analysis that requires 1 second per row to produce an output, such program would take about 10hrs to analyse the entire dataset, and that is something we should keep in mind. That is why, in general, it is a good idea to test large analysis programs on a small sub-set of data and then, once verified that everything runs smoothly, to perform the analysis on the entire dataset.

Let's continue exploring our DataFrame. We have 9 columns, we saw them displayed in our notebook and, luckily enough, their names are pretty descriptive, therefore, in this case, it is quite intuitive to understand what kind of information they contain. It could be useful to store the column names inside a Python variable and to display their names with a corresponding index (this will be useful later).

```
<IPython.core.display.HTML object>
```

```
column_names = df.columns  
print(column_names)  
i=0  
print("Index ) Column name")  
for name in column_names:  
    print(i,")",name)  
    i = i + 1
```

```
Index(['Year of arrival at port of disembarkation', 'Voyage ID', 'Vessel name',  
      'Voyage itinerary imputed port where began (ptdepimp) place',  
      'Voyage itinerary imputed principal place of slave purchase (mjbyptimp) ',  
      'Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place',  
      'VOYAGEID2', 'Captives arrived at 1st port', 'Captain's name'],  
      dtype='object')  
Index ) Column name
```

- 0) Year of arrival at port of disembarkation
- 1) Voyage ID
- 2) Vessel name
- 3) Voyage itinerary imputed port where began (ptdepimp) place
- 4) Voyage itinerary imputed principal place of slave purchase (mjbyptimp)
- 5) Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place
- 6) VOYAGEID2
- 7) Captives arrived at 1st port
- 8) Captain's name

Now we have the column names nicely listed from top to bottom and with their corresponding index assigned to them. You might be tempted to start the indexing from 1, but as in Python the first element of a list (or any other series of elements) has index 0, we started counting from zero. You can obtain the same result with less lines of code, try it out!

```
print("Index) Column name")
for i,name in enumerate(column_names):
    print(f"{i}) {name}")
```

- ```
Index) Column name
0) Year of arrival at port of disembarkation
1) Voyage ID
2) Vessel name
3) Voyage itinerary imputed port where began (ptdepimp) place
4) Voyage itinerary imputed principal place of slave purchase (mjbyptimp)
5) Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place
6) VOYAGEID2
7) Captives arrived at 1st port
8) Captain's name
```

It is now time to figure out what are the rows about. Looking at the column names, we notice that the second one (index 1) is called “Voyage ID”. This indicates that this column contains a specific identifier for the ship voyage, implying that each row contains specific information about a single trip. To verify that each row corresponds to a single voyage, we need to check if all the values of the Voyage ID column are different, i.e. if they are unique.

<IPython.core.display.HTML object>

```
voyage_id = df.iloc[:,1]
print(voyage_id.is_unique)
```



True

We verified that all the values of the Voyage ID column are unique, this means that each of the rows of our DataFrame refers to a single ship voyage. Looking at the other columns, we also notice that information where the voyage began, the port where slaves have been purchased, and the port where slaves have been disembarked is provided. Looking in particular at the fifth column (index 4, “Voyage itinerary imputed principal place of slave purchase”), we notice it contains several NaNs. NaN stands for “Not a Number”, it is a value that appears when something goes wrong in one of the processes ran by our program. If something went wrong, why did not our program stop or told us something about an occurring problem? Because problems may happen more often than you think and if our program stops working everytime it encounters a situation it cannot handle, it would most probably never finish running! In this case, most probably the record does not exist so the data set cell has been filled by NaN, either in our original .csv file or by the `pandas` method `.read_csv()`. NaN are not necessarily something bad, as they can be easily identified and eventually corrected (or simply ignored). Incorrect or missing data may be much harder to spot and correct. In any case, the presence of NaNs or any other missing value can severely affect our data analysis, for this reason before starting analysing the data we need to find and get rid of those values. This process is usually called “data cleaning” and that is exactly what we are going to do in the next chapter.

## 5 Cleaning data

### Previous steps

```
import pandas as pd
data_file = 'data/data.csv'
df = pd.read_csv(data_file)
print(df.shape)
```

(36151, 9)

```
column_names = df.columns
df.head(5)
```

|   | Year of arrival at port of disembarkation | Voyage ID | Vessel name     | Voyage itinerary imputed port |
|---|-------------------------------------------|-----------|-----------------|-------------------------------|
| 0 | 1714.0                                    | 16109     | Freeke Gally    | Bristol                       |
| 1 | 1713.0                                    | 16110     | Greyhound Gally | Bristol                       |
| 2 | 1714.0                                    | 16111     | Jacob           | Bristol                       |
| 3 | 1714.0                                    | 16112     | Jason Gally     | Bristol                       |
| 4 | 1713.0                                    | 16113     | Lawford Gally   | Bristol                       |

Now that we got some familiarity with our dataset, it is time to clean our data, i.e. to get rid of all those NaN values and anything else that might effect our data analysis. Where to start? Well, inspecting the DataFrame by eye, we see several NaNs in the first 5 rows of our DataFrame. The first column we see NaNs is “Voyage itinerary imputed principal place of slave purchase”, the fourth column (index 5). It would be nice to check if also other column have NaNs. Let’s start with the first column, “Year of arrival at port of disembarkation” (index 0), let’s check if this column contains any NaN and then we will repeat the same process for all the other columns.

<IPython.core.display.HTML object>

```
arr_year = df.iloc[:,0]
arr_year_na = arr_year.isna()
print(arr_year_na)
print('Total number of NaNs in the first column:',arr_year_na.sum())
```

```
0 False
1 False
2 False
3 False
4 False
...
36146 False
36147 False
36148 False
36149 False
36150 False
Name: Year of arrival at port of disembarkation, Length: 36151, dtype: bool
Total number of NaNs in the first column: 1
```

```
solution = 'The first column contains 1 NaN value'
question_box(solution=solution)
```

<IPython.core.display.HTML object>

In this way we found out that the first column has 1 NaN (or na) value, that would have been quite hard to spot by eye scrolling 36151 lines! It is great that we found 1 NaN in the first column, but where exactly it is located? What's the corresponding Voyage ID of that value?

<IPython.core.display.HTML object>

```
df[arr_year_na]
```

|       | Year of arrival at port of disembarkation | Voyage ID | Vessel name | Voyage itinerary imputed port v |
|-------|-------------------------------------------|-----------|-------------|---------------------------------|
| 32248 | NaN                                       | 91909     | Kitty       | Liverpool                       |

<IPython.core.display.HTML object>

In this way we can inspect NaNs one by one and we can make a decision about how to handle them. In our DataFrame there are thousands of NaNs (as you will see in a minute) and going through ALL of them one by one is not a good idea. Let's first try to figure out if the other columns have also NaNs and how many are they. The process will be quite straightforward as we already did it for one of the columns, so what we need to do now is to repeat the same procedure for all the other columns.

<IPython.core.display.HTML object>

```
for column_name in column_names:
 selected_column = df[column_name]
 selected_column_na = selected_column.isna()
 n_nan = selected_column_na.sum()
 print(column_name, 'has', n_nan, 'NaN')
```

Year of arrival at port of disembarkation has 1 NaN

Voyage ID has 0 NaN

Vessel name has 1614 NaN

Voyage itinerary imputed port where began (ptdepimp) place has 4508 NaN

Voyage itinerary imputed principal place of slave purchase (mjbyptimp) has 2210 NaN

Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place has 4191 NaN

VOYAGEID2 has 36101 NaN

Captives arrived at 1st port has 17743 NaN

Captain's name has 4028 NaN

and if we want to keep in mind the column index of each column...

```
for i, column_name in enumerate(column_names): \
 print(f"{i}) {column_name} has {df[column_name].isna().sum()} NaN")
```

0) Year of arrival at port of disembarkation has 1 NaN

1) Voyage ID has 0 NaN

2) Vessel name has 1614 NaN

3) Voyage itinerary imputed port where began (ptdepimp) place has 4508 NaN

4) Voyage itinerary imputed principal place of slave purchase (mjbyptimp) has 2210 NaN

5) Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place has 4191 NaN

6) VOYAGEID2 has 36101 NaN

7) Captives arrived at 1st port has 17743 NaN

8) Captain's name has 4028 NaN

<IPython.core.display.HTML object>

At this point we have a general idea of the amount of data missing in our DataFrame. The following question is how to deal with this missing data? There are several things we can do, the easiest option would be just exclude it from our DataFrame. However, in order to answer a research question, we often do not need to use or explore ALL the available information and we would usually be interested in some parameters more than others. In this case our data selection could be performed looking at one or more specific columns. What to do with the rest of the NaNs? We can either leave them as they are and trying to figure out how our analysis program will “digest” these values or find good substitute for them. The value of this substitute will depend on the data type of the columns containing the NaN and on our decision. For example the NaN in the columns containing a descriptive string, like the vessel name or the starting port, could be substituted by the string “unknown”. NaNs in the “Captives arrived [...]” column could be left as they are (you may be tempted to change them to 0, but zero captives is quite different from unknown number of captives) or substituted by, for example, the average of captives during the same year. Each choice will have different implications to our final results, the most important thing in this stage is to clearly document our criteria for filtering NaN. In our specific case we will be mostly interested in the data containing the number of captives, so we want to filter out all those rows where the number of captives is NaN. We will then exclude the column VOYAGEID2 as we already have a voyage ID and it is not listed in the data [variable description](#). To resume, here there are our cleaning criteria:

- All the rows not containing data about the number of captives have been removed;
- All the NaN values in columns with descriptive information (e.g. names) have been substituted with “unknown”;
- The column VOYAGEID2 has been removed from the DataFrame.

<IPython.core.display.HTML object>

```
Display the name of the columns first
print(df.columns)

Select our target columns for cleaning the data
column_to_remove = 'VOYAGEID2'
column_to_remove_nan = 'Captives arrived at 1st port'

Perform Data Cleaning visualising the result step by step
step1, removing column VOYAGEID2 from the DataFrame
cleaned_df_step1 = df.drop(column_to_remove,axis=1)
cleaned_df_step1.head(5)
```

```
Index(['Year of arrival at port of disembarkation', 'Voyage ID', 'Vessel name',
 'Voyage itinerary imputed port where began (ptdepimp) place',
```

```

 'Voyage itinerary imputed principal place of slave purchase (mjbyptimp) ',
 'Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place',
 'VOYAGEID2', 'Captives arrived at 1st port', 'Captain's name'],
 dtype='object')

```

|   | Year of arrival at port of disembarkation | Voyage ID | Vessel name     | Voyage itinerary imputed port |
|---|-------------------------------------------|-----------|-----------------|-------------------------------|
| 0 | 1714.0                                    | 16109     | Freeke Gally    | Bristol                       |
| 1 | 1713.0                                    | 16110     | Greyhound Gally | Bristol                       |
| 2 | 1714.0                                    | 16111     | Jacob           | Bristol                       |
| 3 | 1714.0                                    | 16112     | Jason Gally     | Bristol                       |
| 4 | 1713.0                                    | 16113     | Lawford Gally   | Bristol                       |

```

step2, removing all the rows haveing NaN in the "Captives arrived at 1st port" column
cleaned_df_step2 = cleaned_df_step1.dropna(subset=[column_to_remove_nan])
cleaned_df_step2.head(5)

```

|   | Year of arrival at port of disembarkation | Voyage ID | Vessel name   | Voyage itinerary imputed port w |
|---|-------------------------------------------|-----------|---------------|---------------------------------|
| 0 | 1714.0                                    | 16109     | Freeke Gally  | Bristol                         |
| 2 | 1714.0                                    | 16111     | Jacob         | Bristol                         |
| 3 | 1714.0                                    | 16112     | Jason Gally   | Bristol                         |
| 5 | 1714.0                                    | 16114     | Mercy Gally   | Bristol                         |
| 6 | 1714.0                                    | 16115     | Mermaid Gally | Bristol                         |

```

step3, changing all the other NaN into unknown
cleaned_df = cleaned_df_step2.fillna("unknown")
cleaned_df.head(5)

```

|   | Year of arrival at port of disembarkation | Voyage ID | Vessel name   | Voyage itinerary imputed port w |
|---|-------------------------------------------|-----------|---------------|---------------------------------|
| 0 | 1714.0                                    | 16109     | Freeke Gally  | Bristol                         |
| 2 | 1714.0                                    | 16111     | Jacob         | Bristol                         |
| 3 | 1714.0                                    | 16112     | Jason Gally   | Bristol                         |
| 5 | 1714.0                                    | 16114     | Mercy Gally   | Bristol                         |
| 6 | 1714.0                                    | 16115     | Mermaid Gally | Bristol                         |

```

step4, checking how much data we filtered out
print(cleaned_df.shape)

```

```
n_filtered_rows = len(df)-len(cleaned_df)
per_cent = (n_filtered_rows/len(df))*100
print('We filtered out: ',len(df)-len(cleaned_df),', corresponding to about', round(per_cent,
```

```
(18408, 8)
```

```
We filtered out: 17743 , corresponding to about 49 % of our initial data
```

It seems that because of our filtering, almost half of our data will be excluded from the analysis. This is a quite large percent and we may decide to re-think our filtering criteria to include more data. For example, we could substitute the missing value in the Captives column with an average number of captives per trip. For the purpose of our workshop, we will keep the current filtering criteria and keep our filtered DataFrame as it is.

At this point we obtained a “clean” DataFrame, `cleaned_df`, containing 18408 rows with values organised in 8 columns. We can now start diving deep in the analysis of our DataFrame, we are ready to interrogate this dataset and see which kind of story it is going to tell us.

## 6 Analysing data

### Previous steps

```
import pandas as pd
data_file = 'data/data.csv'
df = pd.read_csv(data_file)
cleaned_df = df.drop('VOYAGEID2',axis=1).dropna(subset=['Captives arrived at 1st port']).f
cleaned_col_names = cleaned_df.columns
cleaned_df.head(10)
```

|    | Year of arrival at port of disembarkation | Voyage ID | Vessel name         | Voyage itinerary imputed |
|----|-------------------------------------------|-----------|---------------------|--------------------------|
| 0  | 1714.0                                    | 16109     | Freeke Gally        | Bristol                  |
| 2  | 1714.0                                    | 16111     | Jacob               | Bristol                  |
| 3  | 1714.0                                    | 16112     | Jason Gally         | Bristol                  |
| 5  | 1714.0                                    | 16114     | Mercy Gally         | Bristol                  |
| 6  | 1714.0                                    | 16115     | Mermaid Gally       | Bristol                  |
| 8  | 1714.0                                    | 16117     | Peterborough        | Bristol                  |
| 9  | 1713.0                                    | 16118     | Resolution          | Bristol                  |
| 10 | 1714.0                                    | 16119     | Richard and William | Bristol                  |
| 11 | 1713.0                                    | 16120     | Rotchdale Gally     | Bristol                  |
| 12 | 1714.0                                    | 16121     | Tunbridge Gally     | Bristol                  |

```
print("Index) Column name")
for i,name in enumerate(cleaned_df.columns):
 print(i,")",name)
```

```
Index) Column name
0) Year of arrival at port of disembarkation
1) Voyage ID
2) Vessel name
3) Voyage itinerary imputed port where began (ptdepimp) place
4) Voyage itinerary imputed principal place of slave purchase (mjbyptimp)
```



- 5 ) Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place
- 6 ) Captives arrived at 1st port
- 7 ) Captain's name

# Data Analysis

It is finally time to ask questions to our data. Let's start with some simple ones regarding the time span of our dataset.

<IPython.core.display.HTML object>

```
arrival_year = cleaned_df.iloc[:,0]
first_year = min(arrival_year)
last_year = max(arrival_year)
year_span = last_year-first_year

print(first_year)
print(last_year)
print(year_span)
```

1520.0  
1866.0  
346.0

```
arrival_year_raw = df.iloc[:,0]
first_year_raw = min(arrival_year_raw)
last_year_raw = max(arrival_year_raw)
year_span_raw = last_year_raw-first_year_raw

print(first_year_raw)
print(last_year_raw)
print(year_span_raw)
```

1514.0  
1866.0  
352.0

<IPython.core.display.HTML object>

We can keep asking questions about numerical values. We focused on time in our last question, let's focus on the number of captives this time.

<IPython.core.display.HTML object>

```
n_captives = cleaned_df.iloc[:,6]
tot_captives = sum(n_captives)
ave_cap_per_voyage = tot_captives/len(cleaned_df)
ave_cap_per_year = tot_captives/year_span
print('Total n. of captives:',tot_captives)
print('Average captives per voyage',round(ave_cap_per_voyage))
print('Average captives per year',round(ave_cap_per_year))
```

```
Total n. of captives: 5082756.0
Average captives per voyage 276
Average captives per year 14690
```

```
filtered_rows = len(df)-len(cleaned_df)
tot_captives_ext = tot_captives + ave_cap_per_voyage*filtered_rows
ave_cap_per_year_adj = tot_captives_ext/year_span_raw
print('Estimated total n. of captives',round(tot_captives_ext))
print('Adjusted average captives per year', round(ave_cap_per_year_adj))
```

```
Estimated total n. of captives 9981894
Adjusted average captives per year 28358
```

<IPython.core.display.HTML object>

So far we computed numbers, but data can be most effectively described using visualization. In our DataFrame we have information about three different locations: the place where the voyage started, the principal port of slave purchase, and the principal port of slave disembarkation. Let's have a closer look at these locations.

<IPython.core.display.HTML object>

```
start_port = cleaned_df.iloc[:,3]
start_port_counts = start_port.value_counts()
```

```
print(type(start_port_counts))
start_port_counts
```

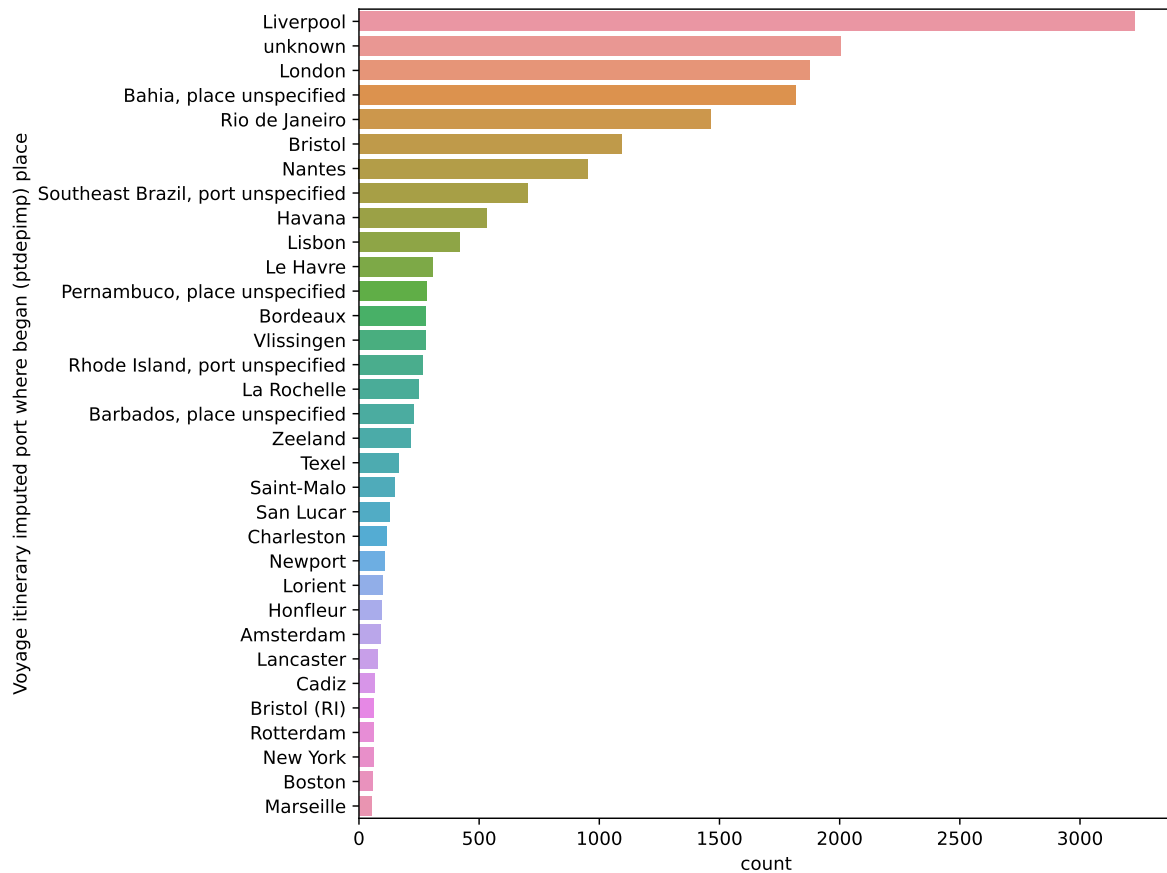
```
<class 'pandas.core.series.Series'>
```

```
Voyage itinerary imputed port where began (ptdepimp) place
Liverpool 3227
unknown 2005
London 1874
Bahia, place unspecified 1815
Rio de Janeiro 1464
...
Mangaratiba 1
Mediterranean coast (France) 1
Canasí 1
Santa Catarina 1
Portland 1
Name: count, Length: 176, dtype: int64
```

```
import seaborn as sns
import matplotlib.pyplot as plt

fig, new_ax = plt.subplots(nrows=1,ncols=1,figsize=(8,8))
filter = start_port_counts > 50
x_data = start_port_counts[filter]
y_data = start_port_counts.index[filter]
sns.barplot(ax=new_ax,x=x_data,y=y_data)
```

```
<Axes: xlabel='count', ylabel='Voyage itinerary imputed port where began (ptdepimp) place'>
```



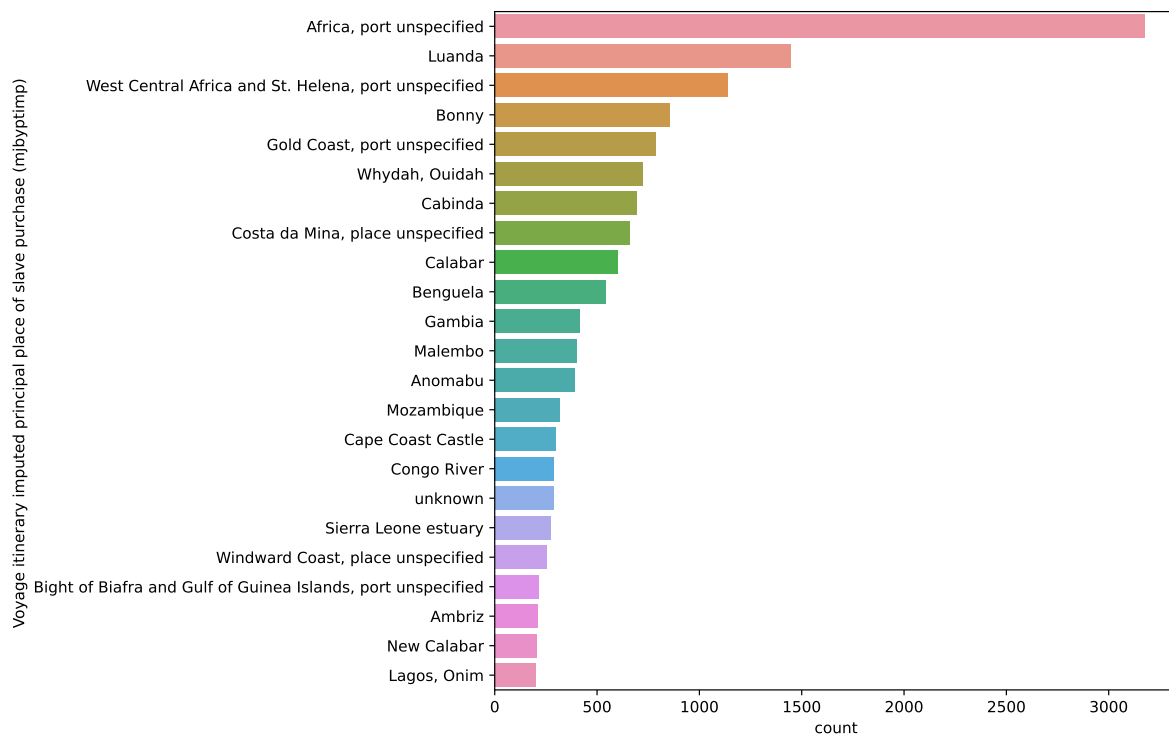
```
main_pur_port = cleaned_df.iloc[:,4]
main_pur_counts = main_pur_port.value_counts()
main_pur_counts
```

```
Voyage itinerary imputed principal place of slave purchase (mjbyptimp)
Africa, port unspecified 3177
Luanda 1447
West Central Africa and St. Helena, port unspecified 1139
Bonny 853
Gold Coast, port unspecified 787
...
Petit Mesurado 1
Eva 1
Pokesoe (Princes Town) 1
Sassandra 1
Sugary (Siekere) 1
```

Name: count, Length: 161, dtype: int64

```
fig, ax = plt.subplots(1,1,figsize=(8,8))
filter = main_pur_counts > 200
sns.barplot(ax=ax,x=main_pur_counts[filter],y=main_pur_counts.index[filter])
```

<Axes: xlabel='count', ylabel='Voyage itinerary imputed principal place of slave purchase (mjbypimp)



```
main_dis_port = cleaned_df.iloc[:,5]
main_dis_counts = main_dis_port.value_counts()
main_dis_counts
```

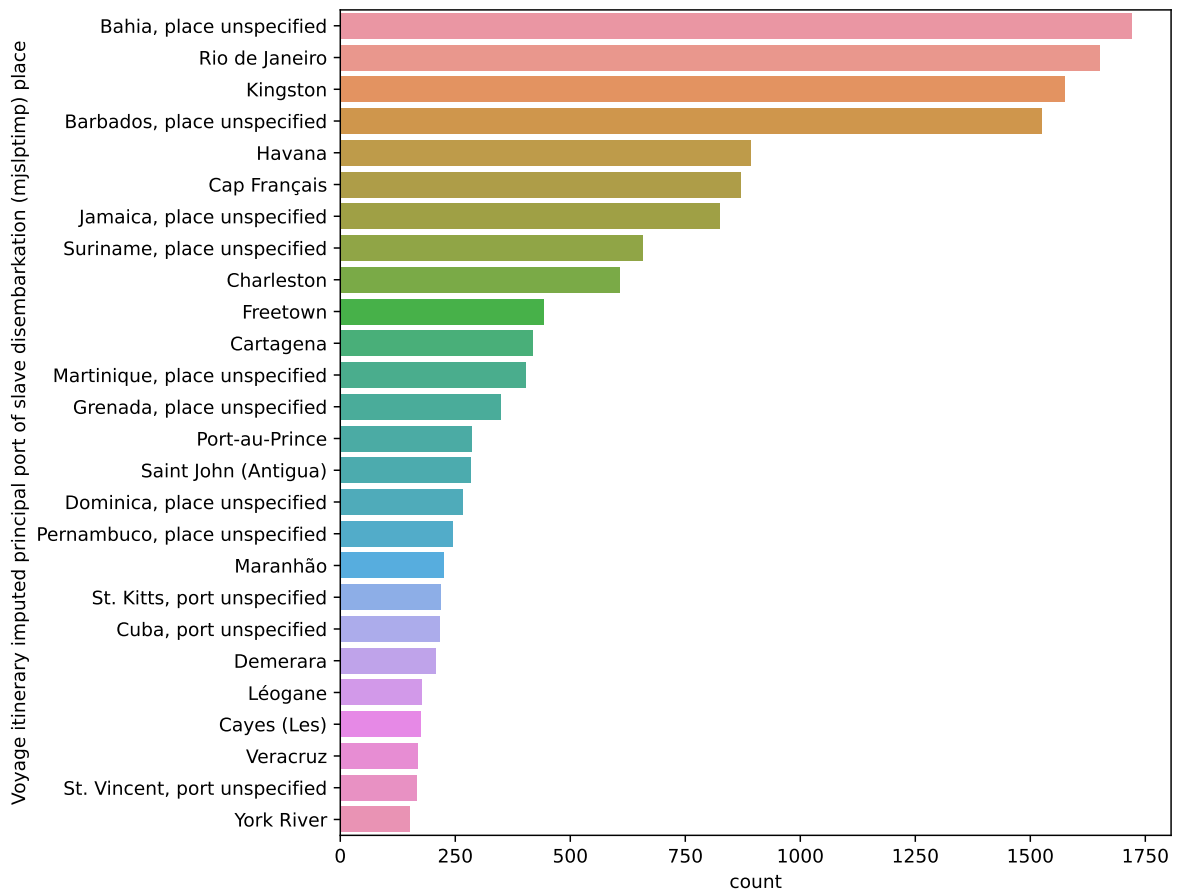
Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place

|                             |      |
|-----------------------------|------|
| Bahia, place unspecified    | 1720 |
| Rio de Janeiro              | 1651 |
| Kingston                    | 1576 |
| Barbados, place unspecified | 1524 |

```
Havana 893
...
France, place unspecified 1
Santa Marta 1
Dois Rios 1
Maceió 1
Bonny 1
Name: count, Length: 240, dtype: int64
```

```
fig, ax = plt.subplots(1,1,figsize=(8,8))
filter = main_dis_counts > 150
sns.barplot(ax=ax,x=main_dis_counts[filter],y=main_dis_counts.index[filter])
```

<Axes: xlabel='count', ylabel='Voyage itinerary imputed principal port of slave disembarkation (mjslptimp) place



<IPython.core.display.HTML object>

Let's try to make now a different type of visualization, a time series, i.e. a plot where we see how parameters change over time

<IPython.core.display.HTML object>

```
col_to_group = 'Year of arrival at port of disembarkation'
col_to_sum = 'Captives arrived at 1st port'
df_per_year = cleaned_df.groupby(col_to_group)[col_to_sum].sum()
print(df_per_year.shape)
df_per_year
```

(298,)

Year of arrival at port of disembarkation

|        |       |
|--------|-------|
| 1520.0 | 44.0  |
| 1526.0 | 115.0 |
| 1527.0 | 46.0  |
| 1532.0 | 589.0 |
| 1534.0 | 354.0 |

...

|        |         |
|--------|---------|
| 1862.0 | 11407.0 |
| 1863.0 | 6739.0  |
| 1864.0 | 3298.0  |
| 1865.0 | 795.0   |
| 1866.0 | 700.0   |

Name: Captives arrived at 1st port, Length: 298, dtype: float64

```
fig, ax = plt.subplots(1,1,figsize=(8,8))
sns.lineplot(ax=ax,x=df_per_year.index,y=df_per_year)
plt.grid()
```

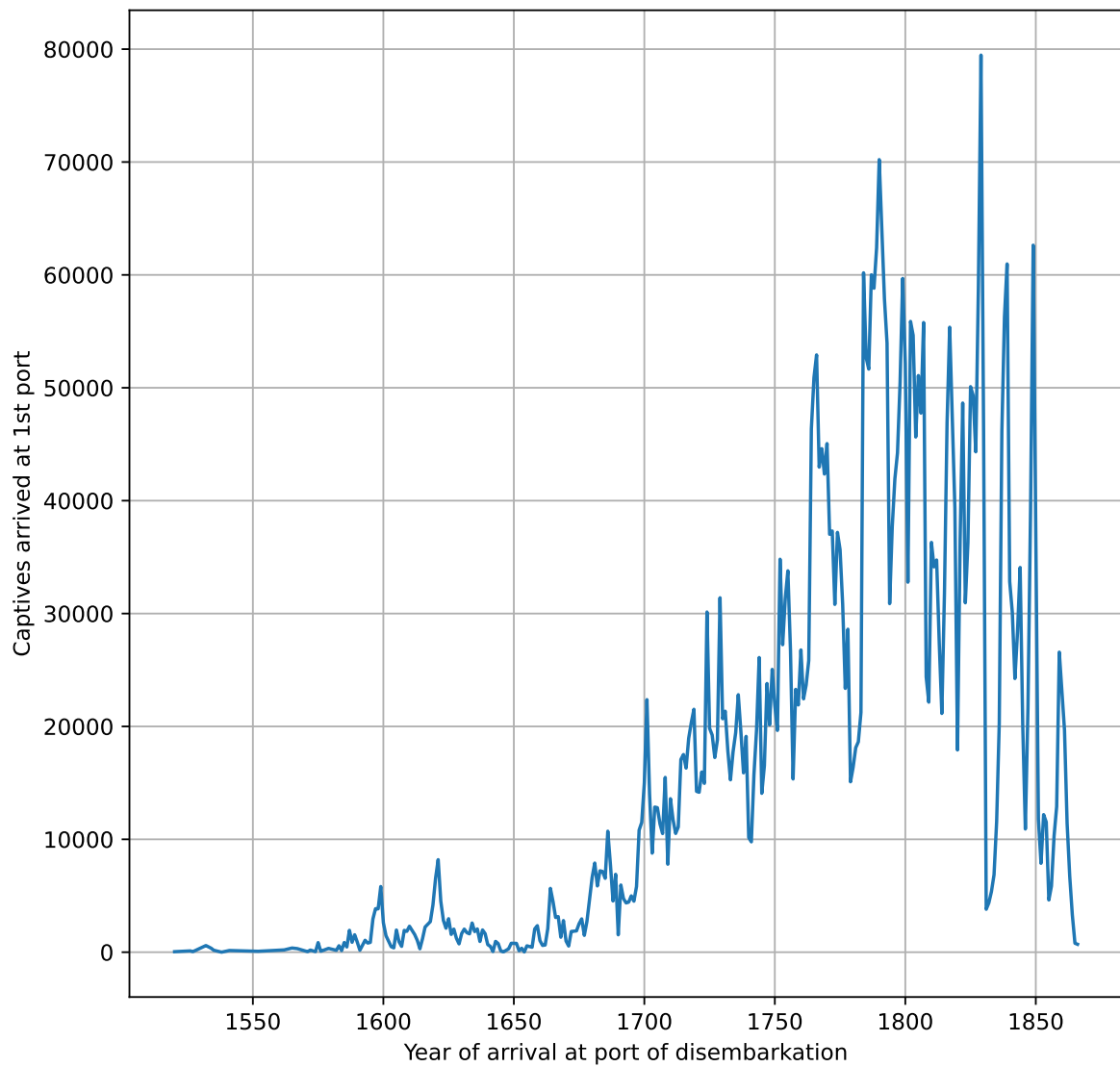
/opt/anaconda3/lib/python3.11/site-packages/seaborn/\_oldcore.py:1119: FutureWarning:

use\_inf\_as\_na option is deprecated and will be removed in a future version. Convert inf values to NaN

/opt/anaconda3/lib/python3.11/site-packages/seaborn/\_oldcore.py:1119: FutureWarning:

use\_inf\_as\_na option is deprecated and will be removed in a future version. Convert inf values to NaN





```
max_index = df_per_year.idxmax()
min_index = df_per_year.idxmin()
min_year = df_per_year[min_index]
max_year = df_per_year[max_index]

print('Min. n. of captives per year:', min_year, 'on', min_index)
print('Max. n. of captives per year:', max_year, 'on', max_index)
```

Min. n. of captives per year: 2.0 on 1538.0

Max. n. of captives per year: 79472.0 on 1829.0

```
solution = 'The total number of captives is almost constant up to 1650, with the exception
and 1622. The number increases steadily up to 1800 and decreases afterwards. The times ser
by low and high peek. The number of captives per year reaches its maximum on 1829 with alm
that year. The minimum is 2 captives on 1538.'
question_box(solution=solution)
```

<IPython.core.display.HTML object>

Time series are very interesting to describe the trends of phenomema at different scale. Our plot ticks are separated by 50 years, this is fine to visualise trends over centuries, but we cannot see what's happening on decades.

<IPython.core.display.HTML object>

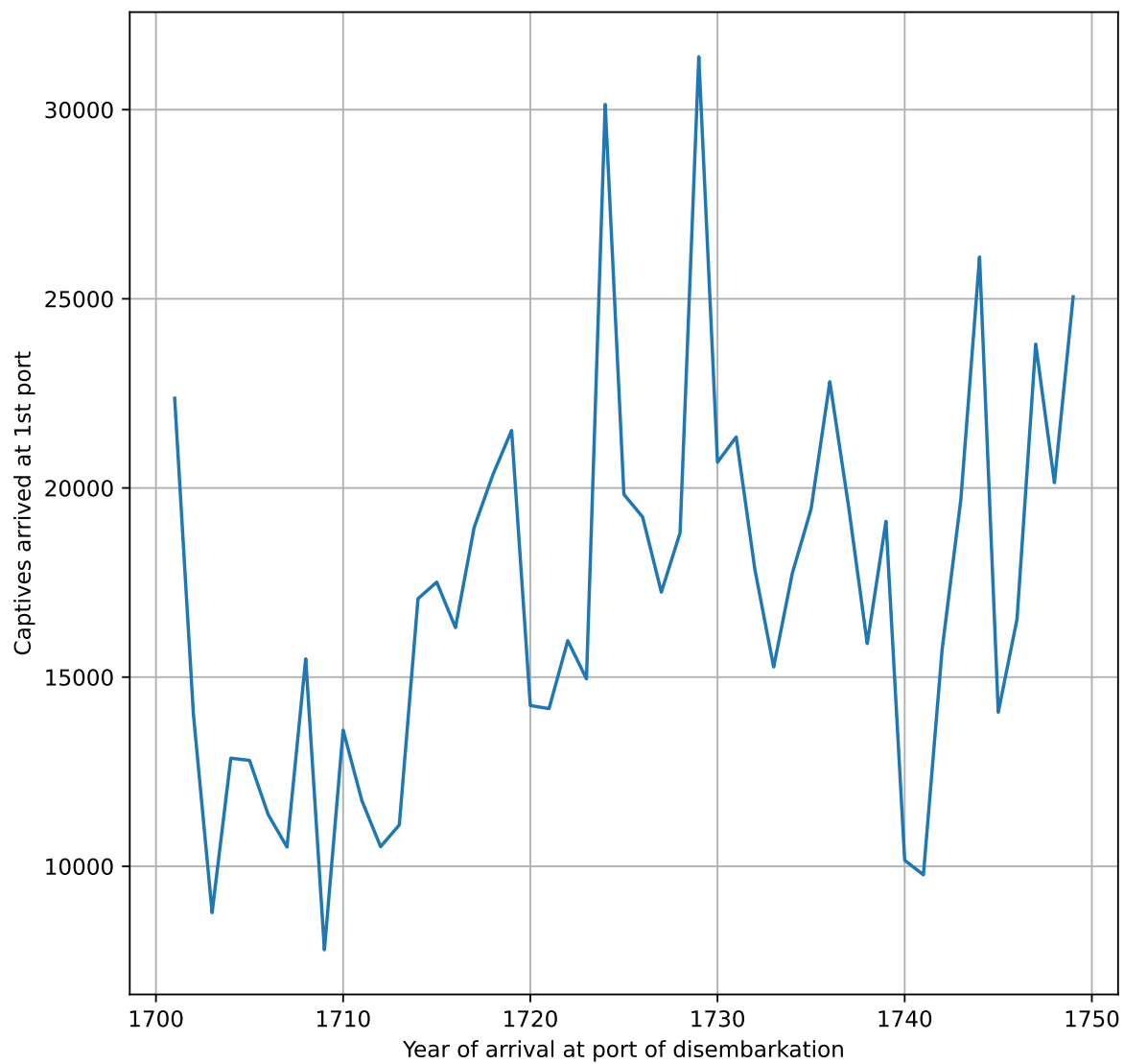
```
time_filter = (df_per_year.index > 1700) & (df_per_year.index < 1750)
fig, ax = plt.subplots(1,1,figsize=(8,8))
x_data = df_per_year.index[time_filter]
y_data = df_per_year[time_filter]
sns.lineplot(ax=ax,x=x_data,y=y_data)
plt.grid()
```

/opt/anaconda3/lib/python3.11/site-packages/seaborn/\_oldcore.py:1119: FutureWarning:

use\_inf\_as\_na option is deprecated and will be removed in a future version. Convert inf values to None

/opt/anaconda3/lib/python3.11/site-packages/seaborn/\_oldcore.py:1119: FutureWarning:

use\_inf\_as\_na option is deprecated and will be removed in a future version. Convert inf values to None



# Summary

## Databases and Data Analysis

In the first part of the workshop, we conducted an interactive exercise to experience the main challenges associated with converting unstructured data into structured, organized, tabular (and generally more mathematical) data. Having structured, machine-readable data is essential for proper data analysis. It is important to note that database creation, which marks the beginning of every data analysis process, can be affected by errors, missing data, and biases driven by human decisions.

In the second part of the workshop, we focused on data analysis. Data analysis is often only considered in relation to programming tools. However, we aimed to emphasize the fundamental principles of data analysis, emphasizing that analyzing data essentially involves querying a dataset. While some information may be easy to retrieve, other information may be hidden or require assumptions and speculation. Python (or any other programming language) is simply a tool to translate our questions into a machine-readable form.

Regardless of our data analysis process and the tools we use, it is crucial to describe and document our choices so that other researchers can reproduce the entire workflow that led to our conclusions.

## General data analysis workflow

### 1. Define a Research question:

- Understand the problem or question you are trying to address;
- Clearly define the goals, objectives, and sub-task to answer a research question.

### 2. Collect/Organise Data:

- Collect relevant data from various sources;
- Ensure data quality, address any missing or inconsistent data, ensure proper data structure.

### 3. Clean Data:

- Clean and preprocess the data to handle missing values, outliers, and errors;

- Standardize or normalize data formats if necessary.

**4. Explore Data:**

- Explore the data using statistical and visual methods;
- Identify patterns, trends, and relationships in the data.

**5. (Model):**

- Select appropriate models based on the analysis goals;
- Evaluate the model's performance using metrics relevant to the analysis;
- Fine-tune the model if necessary.

**6. Interpret Data:**

- Interpret the results of the analysis in the context of the initial research question;
- Draw conclusions and make recommendations based on the findings.

**7. Visualization and Reporting:**

- Create visualizations to communicate key findings;
- Prepare a comprehensive report summarizing the analysis process, results, and insights.

## What's next?

Congratulations! If you are reading these few lines you survived our workshop on analysing cultural data (and you even went through the documentation!). Our workshop was only an introduction, it would have been impossible to cover everything related to analysing cultural data in only four hours, but we hope you now have a general overview of how data analysis is performed and of all the caveats related to data base creation and analysis.

What's next? You can build on top of what we have discussed during the workshop. Here there is a list of possible further steps, good luck!

- **More data:** try to obtain more data in .csv form and perform the data analysis on this new data. You can either get new data in the [SlaveVoyages website](#) or download any data in .csv format. Just remember to download it in the “data” directory and to change `data_file` into “data/your\_file.csv”. You can also use data in a different format (like excel sheet for example) and read it with the corresponding `pandas` tool;
- **Learning more about Python:** if you know nothing about programming and Python, you might consider to invest some time for learning about it. The Utrecht University Library and the Centre for Digital Humanities (CDH) offer free Python courses: have a look at the Research Data Management (RDM) workshop [page](#) and at the CDH workshop [page](#);
- **More data questioning:** you can ask your own questions to data and find a way to implement that in Python (or any other programming language you are going to use)

# Glossaries

## Data Glossary

### Controlled vocabularies

Standardised sets of terms or phrases used to ensure consistency and accuracy in categorising and retrieving information.

### Data cleaning

The process of identifying and correcting errors, inconsistencies, and inaccuracies in a dataset to improve its quality and reliability.

### Data harmonisation

The process of integrating and standardising data from different sources or formats to ensure consistency and compatibility for analysis or other purposes.

### Data models

Abstract representations defining the structure, relationships, and constraints of data within a system or database.

### Enrichment

The process of enhancing or augmenting existing data with additional information to improve its quality, usability, or value.

### Graph database

A database structured around graph theory, where data entities are represented as nodes and their relationships as edges, facilitating complex and interconnected data querying.

## **ID**

Identification or identifier used to uniquely distinguish an entity within a system.

## **Normalisation**

The process of organising data in a database to reduce redundancy and dependency by dividing large tables into smaller ones and defining relationships between them.

## **Relational database**

A type of database management system (DBMS) organised around tables and relationships, adhering to the principles of the relational model.

## **Programming glossary**

### **Code**

Code is like a set of instructions that tells the computer what to do. It's similar to a recipe for the computer. The instructions for a specific task may vary according to the programming language you use, that is why you also usually specify the language you are using (e.g. "Python code").

### **csv**

CSV (coma separated values) is a way to store information, like making lists. It's a simple way to organize data, like names and ages, using commas. A file containing data organised in this way, has usually the extension ".csv". Even if the word "coma" is present in the acronym, data can be also separated by other symbols such as ";" or ":" and still be contained in a .csv file.

### **DataFrame**

A DataFrame is a Python object included in the pandas library. It is basically a table where information is organised in rows and columns. Every DataFrame row has an index that can be either a numeric value or a string (i.e. a label)



## Initialisation

Initialise basically means getting things ready. It's the starting point before using something in a program. Initialising a variable, in particular, means assigning a value to it for the first time:

```
We initialise the variable name and age for the first time with a string (a word) and a
name = 'Stefano'
age = 28
```

## Library

See package

## Loop

A loop is like a computer doing something over and over again. It's a way to repeat a task multiple times.

## Object

An object is like a thing in the computer's world. It has characteristics and things it can do. For example, a dog can have a name and bark.

## Package

A library or package is like a toolbox with ready-made tools. It's a collection of helpful code (objects and functions) that programmers can use to make their work easier. Packages are made by the programming community and are usually organised according to certain specific tasks. You may find packages specific for time series analysis, text analysis, satellite image analysis, etc. The advantage of using a package is that you do not have to spend time and energy in finding solutions to problems already tackled by other people. Packages do not usually come automatically with the basic programming language installation (so to optimize space), but need to first be downloaded and imported. You may think at packages as building tools. Downloading a package would be similar to buying them from the shop and importing them is similar to get them ready to work (you do not need all your tools ALL the time for ANY house job, right?)

## **Series**

A Series is a DataFrame with a single column. Like DataFrames, a Series is an object belonging to the pandas library. Series rows, like in a DataFrame, have indices that can be either values or labels.

## **Variable**

A variable is like a box where you can keep and change information. It is a container for numbers or words.

# Resources

## Research Support

- Utrecht University Library [website](#)
- Centre for Digital Humanities (CDH) [website](#)

## Datasets

- **SlaveVoyages:** collaborative digital initiative that compiles and makes publicly accessible records of the largest slave trades in history. [link](#)
- **Cooper Hewitt collection:** over 30 Centuries of design online, from ancient Roman marble to Pre-Columbian textiles, Renaissance drawings, contemporary 3D-printed chars, and digital code. [link](#)
- **KU Libraries Special Collections:** portraits dataset of descriptive metadata representing all the single-file documents with graphic art from the Special Collections of the KU Leuven Libraries that depict people. [link](#)
- **The Magister Dixit Collection (Leuven):** 552 manuscripts of lecture notes of the Old University of Leuven (1425-1797). [link](#)
- **Getty-Knoedler Stock Books:** more than 40,000 records transcribed from the 11 painting stock books and enhanced with information from the 21 paintings and water-colors sales books of M. Knoedler & Co. in New York (1872-1970). [link](#)
- **Pantheon Project: Historical Popularity Index:** The Pantheon 1.0 data measures the global popularity of historical characters according to the metrics of their Wikidata pages in different languages. [link](#)
- **National Gallery of Art Open Data:** some of the National Gallery of Art (Washington D. C.) database made public for exploration and educational purposes. [link](#)

## Programming

- Python [webpage](#);
- Pandas (Python Data Analysis package) [webpage](#);
- Matplotlib (Python visualization package) [webpage](#);
- Seaborn webpage (Python visualization package) [webpage](#).